

西北工業大學

NORTHWESTERN POLYTECHNICAL UNIVERSITY

NPUcore-BLOSSOM 设计文档

队长：沈天乐

队员：李惟鑫, 杨峥巍

指导老师：张羽

2025 年 8 月 17 日

目录

1 引言 **2**

2 基准 OS 及增量说明 **2**

2.1 RISC-V:*NPUcore+* 2

2.2 LoongArch64:*NPUcore-IMPACT!!!* 2

2.3 RISC-V&LoongArch64:*NPUcore-BLOSSOM* 3

3 框架模块介绍 **4**

3.1 平台层 4

3.2 硬件抽象层 5

3.2.1 arch 模块 6

3.2.2 config 模块 7

3.2.3 platform 模块 7

3.3 内核层 7

3.3.1 进程管理 7

3.3.2 内存管理 7

3.3.3 文件系统 8

3.3.4 系统调用 8

3.4 应用层 8

3.4.1 各赛段测评成绩 8

3.4.2 额外应用支持 9

4 其他 **11**

4.1 文档仓库和介绍 11

5 致谢 **11**

1 引言

欢迎来到 NPUcore-BLOSSOM[2] 的设计文档！这个文档详细说明了 NPUcore-BLOSSOM 的 baseline 选取、增量说明及 OS 内核从顶层到底层的框架设计。其中, 硬件抽象层 (Hardware Abstract Layer) 是按照大赛要求新增的一个中间层, 通过这层抽象,NPUcore 从以前单一指令集内核转变成为了支持 LoongArch64 及 RISC-V 两种指令集的内核, 大大提高了 NPUcore 的扩展性; 最后的应用层会介绍我们在大赛各个阶段的评测及额外支持的应用。

2 基准 OS 及增量说明

2.1 RISC-V:*NPUcore+*

NPUcore+[3] 是西北工业大学 2023 年参加操作系统内核赛的二等奖作品, 它支持的是 RISC-V 指令集, 拥有**系统调用、内存管理、进程管理**和**文件系统**等核心功能, 其中文件系统支持 FAT32。以下是其内核架构图:

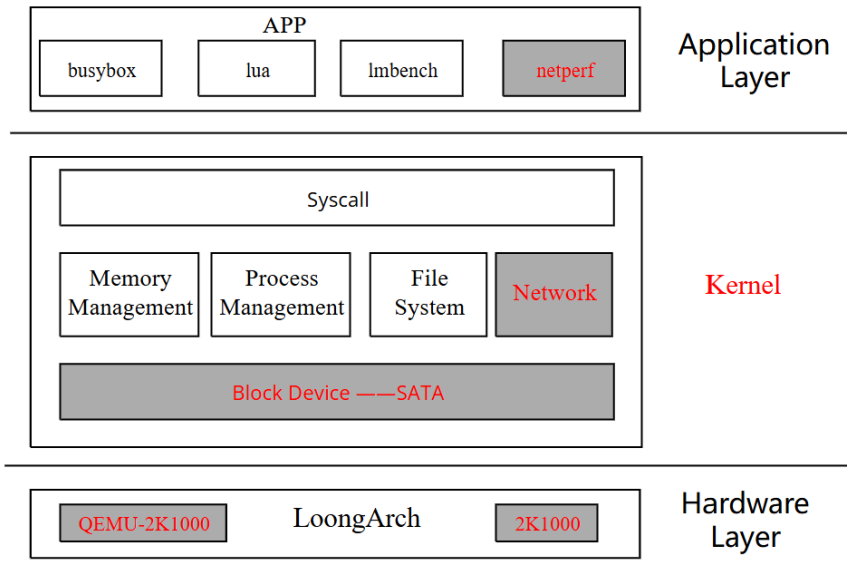


图 1: NPUcore+ 内核架构图

2.2 LoongArch64:*NPUcore-IMPACT!!!*

NPUcore-IMPACT!!![4] 是西北工业大学 2024 年“全国大学生计算机系统能力大赛 - 操作系统设计赛 (全国)-OS 内核实现赛道龙芯 LA2K1000 分赛道”**一等奖**（无特等奖）参赛作品。NPUcore-IMPACT 是一个使用 Rust 编写的基于 LoongArch 架构的

类 Unix 操作系统。基于原先的 NPUcore(RISCV) 开发迭代并扩展, 实现 POSIX 标准系统调用 90 个, 支持信号机制及线程。以下是其内核架构图:

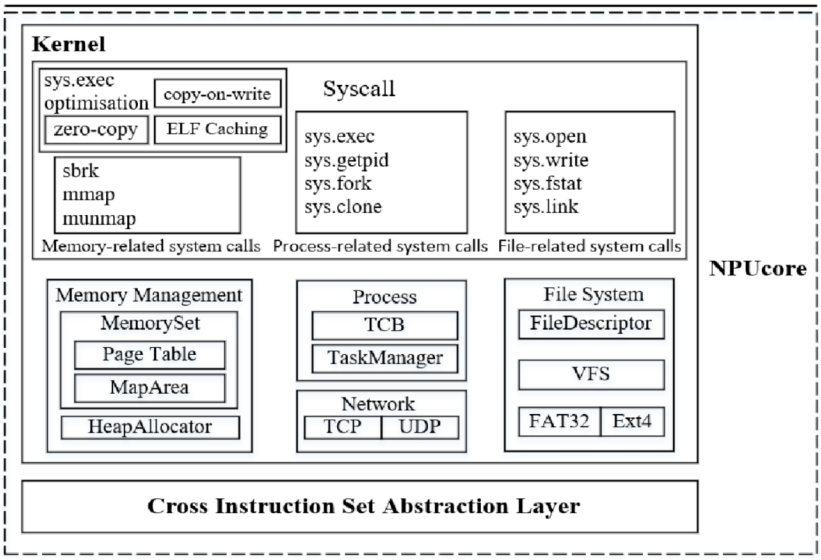


图 2: NPUcore-IMPACT!!! 内核架构图

2.3 RISC-V&LoongArch64:*NPUcore-BLOSSOM*

NPUcore-BLOSSOM 作为 2025 年操作系统内核赛的参赛作品, 在吸取了以上两个优秀的"NPUcore 家族" 作品的经验后, 拥有以下特点:

- 完善、支持了 EXT4 文件系统
- 实现 HAL 层进行双架构支持
- 运行更多的应用

下面是我们的内核架构图:

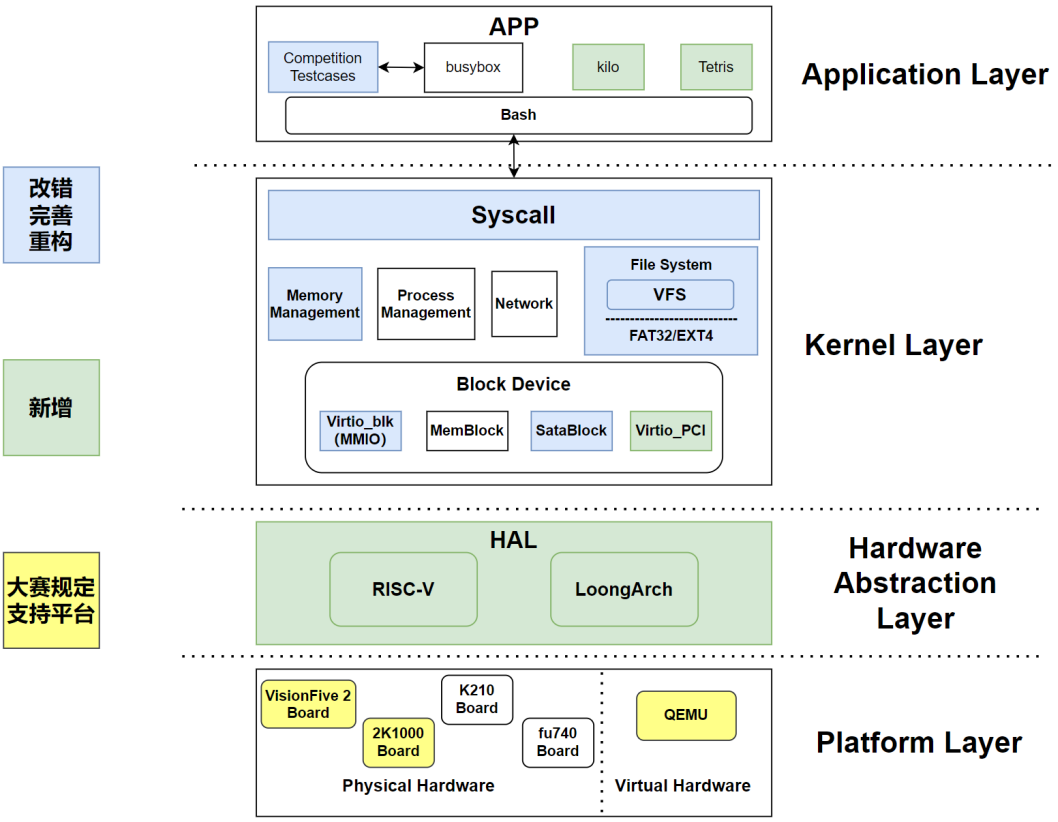


图 3: NPUcore-BLOSSOM 架构图

3 框架模块介绍

下面依次介绍各个模块层次的内容。

3.1 平台层

平台层是系统的基础支撑，为上层提供了硬件资源和运行环境。NPUcore 系列及 NPUcore-BLOSSOM 目前支持以下各类平台 (红色为本次大赛要求平台):

表 1: NPUcore 运行平台及架构支持

平台类型	具体平台	LoongArch64	RISC-V
虚拟平台	QEMU	√	√
物理平台	VisionFive2 Board	-	√
	2K1000 Board	√	-
	K210 Board	-	√
	fu740 Board	-	√

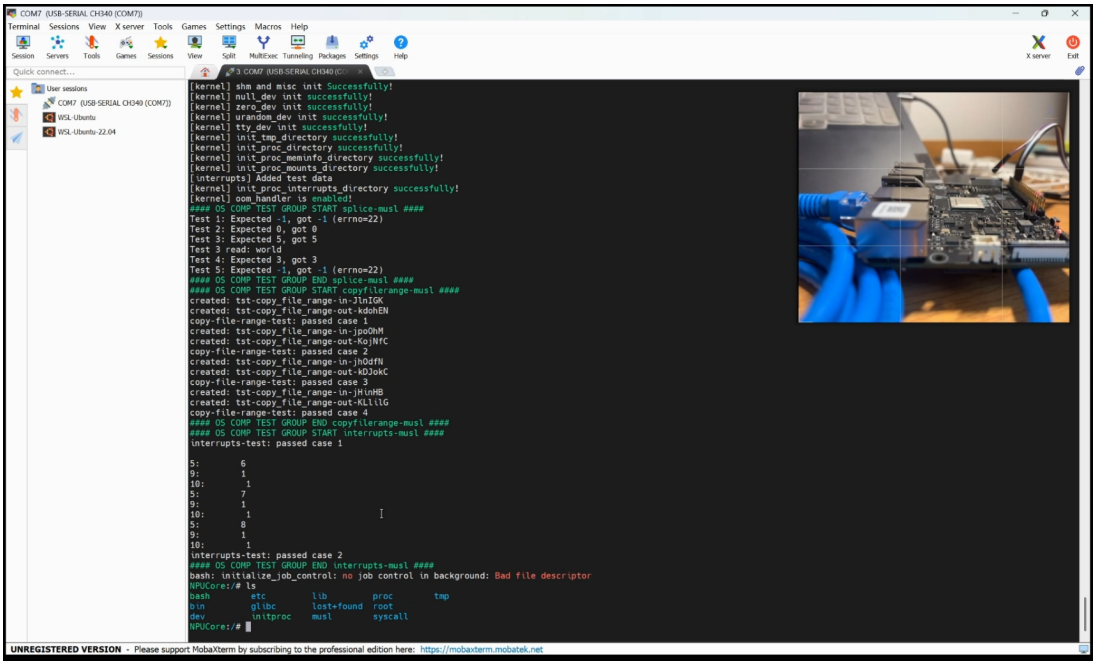


图 4: VisionFive2 Board 展示

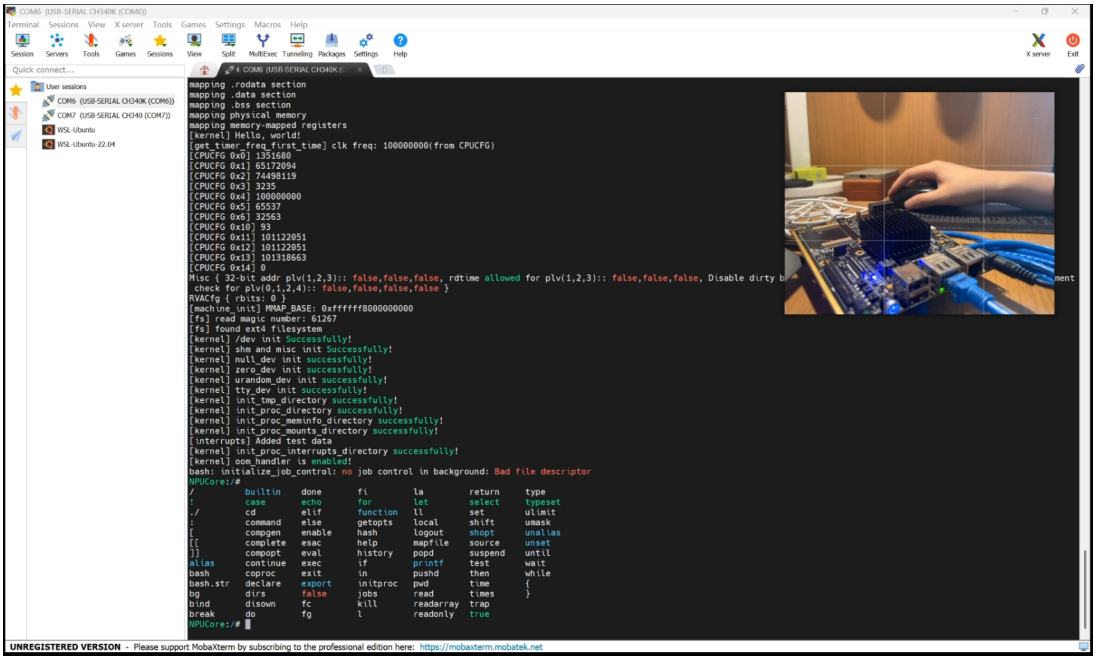


图 5: 2K1000 Board 展示

3.2 硬件抽象层

我们实现了一个硬件抽象层，这个抽象层提供了一个统一的接口来访问底层硬件资源，使得上层软件可以不依赖于具体的硬件实现，从而提高代码的可移植性和可维护性。

该抽象层的核心由架构，平台还有相应配置文件组成，其核心部分为架构和平台两个模块。HAL 层的工作树如下 (* 为省略其中的文件)：

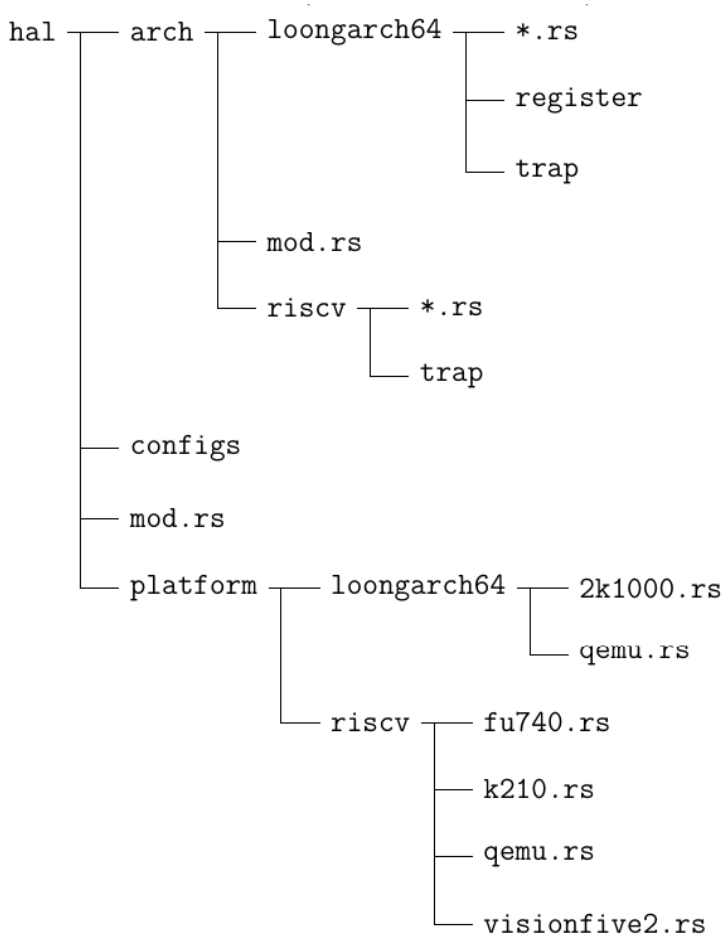


图 6: HAL-tree

3.2.1 arch 模块

arch 模块实现了不同的处理器架构，分别实现了 loongarch64 和 riscv 两种处理器架构，架构文件通过调用 platform 模块对应的硬件平台来实现内核的运行。

loongarch 架构： 该模块实现了与龙芯 64 位架构相关的各种硬件操作和配置，涵盖从寄存器操作到陷阱处理、内存管理、上下文切换等多个方面的内容。采用模块化设计，将寄存器操作、中断异常处理在不同工作目录下分开处理，方便后续开发。其中，register 目录包含架构的寄存器操作相关代码，trap 目录包含 trap 处理相关代码。

riscv 架构： 该模块文件共同为上层软件提供与 RISC-V 硬件架构相关的底层支持，实现了 RISC-V 架构中的异常处理、内核栈相关问题处理、SV39 分页机制、上下文切

换及定时器相关功能。

3.2.2 config 模块

该模块下包含两个配置文件（loongarch64-qemu-virt.toml 和 riscv64-qemu-virt.toml），它们为特定架构在 QEMU 虚拟环境下的运行提供参数设置:- loongarch64-qemu-virt.toml：针对龙芯架构，适用于在 QEMU 模拟器提供的虚拟环境中运行基于此架构的系统或程序。- riscv64-qemu-virt.toml：专门为 RISC-V 64 位架构设计，用于在 QEMU 虚拟环境下配置相关参数，使基于 RISC-V 架构的系统能够在模拟环境中正常启动和运行。

3.2.3 platform 模块

本模块是适配不同硬件平台的关键部分，它会根据具体的硬件架构和型号，提供对应的底层操作接口，以屏蔽硬件差异，让上层软件能以统一的方式与硬件交互。

loongarch 平台： loongarch64 子模块是针对龙芯 LoongArch64 架构的平台适配部分：- 2k1000.rs：用于适配龙芯 2K1000 型号的芯片。- qemu.rs：针对在 QEMU 模拟器中模拟的 LoongArch64 环境进行适配。

riscv 平台： riscv 子模块是针对 RISC-V 架构的平台适配部分，包含多个不同 RISC-V 平台的适配文件：- fu740.rs：用于适配 FU740 芯片。- k210.rs：适配 K210 芯片。- qemu.rs：针对 QEMU 模拟器中模拟的 RISC-V 环境进行适配。- visionfive2.rs：适配 VisionFive2 开发板。

3.3 内核层

内核层是系统的核心，负责管理系统的进程、内存、文件系统等关键资源，以及提供进程间通信、中断处理等核心功能。内核层的设计直接影响系统的性能和稳定性。

3.3.1 进程管理

详见https://gitee.com/differential1012/npucore-blossom-docs/blob/master/Doc/finalDoc_kernel/%E8%BF%9B%E7%A8%8B%E7%AE%A1%E7%90%86/3.3.1%20%E8%BF%9B%E7%A8%8B%E7%AE%A1%E7%90%86.md

3.3.2 内存管理

详见https://gitee.com/differential1012/npucore-blossom-docs/blob/master/Doc/finalDoc_kernel/%E5%86%85%E5%AD%98%E7%AE%A1%E7%90%86/3.3.2%20%E5%86%85%

E5%AD%98%E7%AE%A1%E7%90%86.md

3.3.3 文件系统

详见https://gitee.com/differential1012/npucore-blossom-docs/blob/master/Doc/finalDoc_kernel/%E6%96%87%E4%BB%B6%E7%B3%BB%E7%BB%9F/3.3.3%20%E6%96%87%E4%BB%B6%E7%B3%BB%E7%BB%9F.md

3.3.4 系统调用

详见https://gitee.com/differential1012/npucore-blossom-docs/blob/master/Doc/finalDoc_kernel/%E7%B3%BB%E7%BB%9F%E8%B0%83%E7%94%A8/3.3.4%20%E7%B3%BB%E7%BB%9F%E8%B0%83%E7%94%A8.md

3.4 应用层

应用层是系统与用户交互的接口，包括各种应用程序和服务，如办公软件、游戏、网络服务等。应用层基于内核层提供的功能和接口，实现具体的业务逻辑，满足用户的不同需求。NPUcore-BLOSSOM 在实现了较为完整的系统调用和各类核心内核功能的基础上, 能通过若干测试程序并且支持了额外的轻量应用。

3.4.1 各赛段测评成绩

表 2: 初赛测评结果

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	51	10	54	53	168
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	0.0	0.0	0.0	0.0	0.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	0.0	0.0	0.0	0.0	0.0
libctest	-	-	185	185	370
lmbench	35.393361764714804	0.0	26.865222609476536	28.552453560873833	90.81103793506517
ltp	0	0	0	0	0
lua	9	0	9	9	27
netperf	0.0	0.0	0.0	0.0	0.0
总分	197.3933617647148	112.0	376.8652226094765	377.55245356087386	1063.811037935065

表 3: 决赛阶段 1

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
copyfilerange	7.5	7.5	7.5	7.5	30.0
interrupts	3.75	3.75	3.75	3.75	15.0
splice	10	10	10	10	40
总分	21.25	21.25	21.25	21.25	85.0

表 4: 决赛阶段初赛重测

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	49	48	54	53	204
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	0.0	0.0	0.0	0.0	0.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	0.0	0.0	0.0	0.0	0.0
libctest	-	-	185	185	370
lmbench	35.539892441278674	36.2748649755899	26.954781536985646	28.41534870827281	127.18488766212704
ltp	0	0	316	319	635
lua	9	9	9	9	36
netperf	0.0	0.0	0.0	0.0	0.0
总分	195.53989244127868	195.2748649755899	692.9547815369856	696.4153487082729	1780.184887662127

3.4.2 额外应用支持

NPUcore-BLOSSOM 支持多种基于终端的实用应用，以下为其中两个已经适配的例子：

- **kilo 文本编辑器**
  集成了由 Salvatore Sanfilippo（antirez）开发的轻量级终端文本编辑器（项目地址：`https://github.com/antirez/kilo`）。该编辑器具有以下特点：
  - 极简设计：核心代码不足 1000 行，无需依赖 curses 等外部库，仅通过 VT100 终端转义序列实现交互
  - 基础功能：支持文件创建/编辑/保存（快捷键 CTRL+S）、退出（CTRL+Q）及字符串搜索（CTRL+F）
  - 扩展价值：源码以单一文件（kilo.c）形式组织，遵循 BSD 2 条款许可，可作为终端交互程序开发的参考框架

- **终端版俄罗斯方块（Tetris）**
  支持 S1M0N38 开发的终端俄罗斯方块实现（项目地址：`https://github.com/S1M0N38/tetris`），其核心特性包括：
  - 技术实现：采用 C++ 编写，基于 ncurses 库处理终端界面渲染，使用扩展 ASCII 字符构建游戏元素 (因为 ncurses 库适配难度高, 我们对项目进行了改造使其直接用字符进行运行)
  - 游戏机制：遵循经典规则，包含 7 种标准方块（Tetrominoes）、超级旋转系统（SRS）及等级进阶机制
  - 交互设计：支持方向键控制方块移动/旋转 (改造后用 WASD)，CTRL+C 强制退出，帧率限制为 10 FPS 以适配终端环境

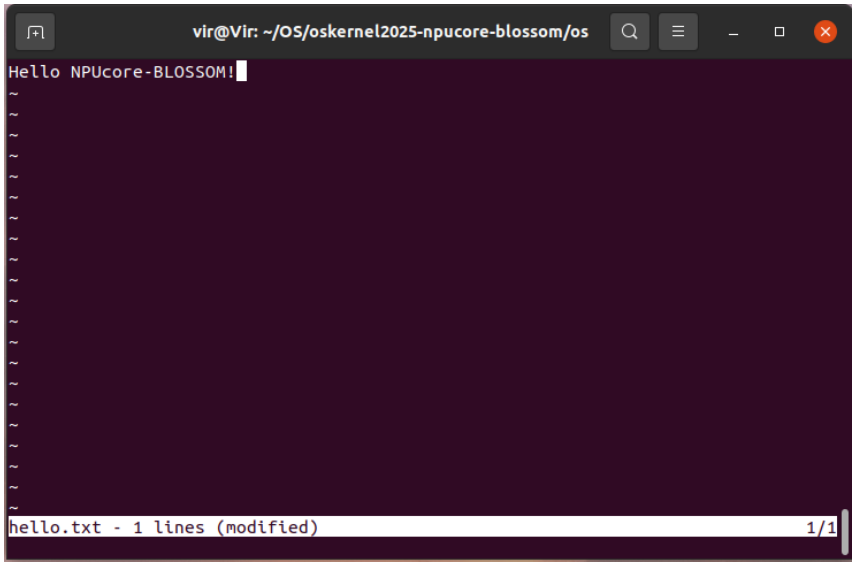


图 7: kilo 展示

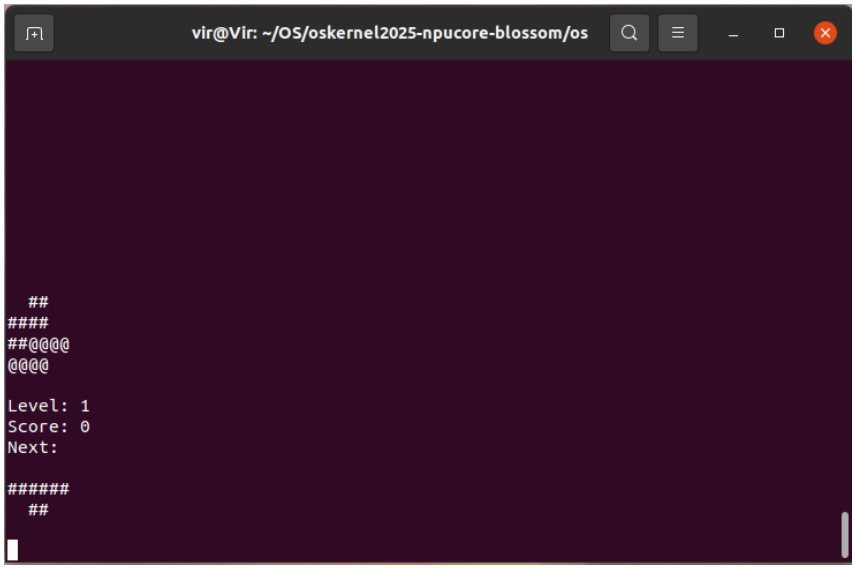


图 8: Tetris 展示

4 其他

4.1 文档仓库和介绍

本项目的相关文档与资料均托管于码云（Gitee）仓库，仓库地址为：`https://gitee.com/differential1012/npucore-blossom-docs`。该仓库系统整理了项目开发过程中的各类文档资源，便于团队协作与项目追溯，主要包含以下核心目录及内容：

- 2week_report：存放定期汇报材料，用于向指导老师张羽同步项目进展，内容涵盖阶段性开发成果、遇到的技术难点、解决方案及后续计划，形成规范化的开发闭环。
- Doc：集中管理初赛与决赛阶段的各类核心文档，演示 PPT 等，完整记录了从项目启动到各阶段验收的关键资料。
- TODO：维护未解决的 bug 清单
- develop：记录日常开发日志
- 平台测评输出留档：专门保存初赛阶段的测评数据

5 致谢

在 NPUcore-BLOSSOM 的设计与实现过程中，得到了指导老师张羽的悉心指导和帮助，老师的专业知识和丰富经验为我们提供了宝贵的建议和思路。同时，也感谢团队

成员之间的紧密合作和相互支持，使得项目能够顺利完成。在此，衷心感谢同为本次大赛参赛选手，来自广告位招租的**林哥**对我们实现 HAL 和 EXT4 的帮助；还有上一届参赛选手，来自重生之我是菜狗的**郭学长**对我们进行上板的指导。

参考文献

[1] 张羽, 郭晥. 基于 Rust 语言的 NPUcore 操作系统内核构建实践 [M]. 西北工业大学出版社, 2024.

[2] OSKernel2025-NPUcore-BLOSSOM[EB/OL].https://gitlab.eduxiji.net/T202510699995278/oskernel2025-npucore-blossom.

[3] oskernel2023-npucore-plus[EB/OL].https://gitlab.eduxiji.net/202310699101073/oskernel2023-npucore-plus/-/tree/master?ref_type=heads.

[4] NPUcore-IMPACT!!![EB/OL].https://github.com/Fediory/NPUcore-IMPACT/tree/NPUcore-FF